a. The 2 generative AI's used were **Claude's Opus 5.5 (Medium)** and **Gemini's 3.8 Flash (Low)**.

b. They were each accessed in their native agent application for macOS, being **Claude Code** and **Google Antigravity** respectively.

c. Identical **prompt** handed off to both agents:

Create the following program as detailed:
Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• A typical puzzle is shown here (image credit: Wikimedia Commons):
o
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively

try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt.

d. Generated code
   i. Claude’s initial result (commented)

```
/*
EECS 348 Assignment 4
Sudoku solver: Ingest a 9x9 sudoku puzzle and return all solutions
Input: puzzle1.txt thru puzzle5.txt. Or dedicated filenames
Output: Filename, the original board, errors, and the possible solutions
Claude Opus 5.5 (Medium)
Matthew Bisbee
10/15/2026
*/

#include <array>
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

class SudokuGrid
{
public:
   // Use nine rows and nine columns
   static constexpr int SIZE = 9;

   // Start with an empty board by filling each row with zeros
   SudokuGrid()
   {
       for (auto &row : cells_)
       {
           row.fill(0); // Zero = blank tile
       }
   }

   // Return stored value at that location
   int get(int row, int col) const
   {
       return cells_[row][col];
   }
   void set(int row, int col, int value)
   {
       cells_[row][col] = value;
   }
```

```
   // Print each row in order
   void print(std::ostream &out) const
   {
       for (int r = 0; r < SIZE; ++r)
       {
           for (int c = 0; c < SIZE; ++c)
           {
               if (c > 0)
               {
                   out << ' ';
               }
               int v = cells_[r][c];
               if (v == 0)
               {
                   out << '_';
               }
               else
               {
                   out << static_cast<char>('0' + v);
               }
           }
           out << '\n';
       }
   }

private:
   std::array<std::array<int, SIZE>, SIZE> cells_;
};

// Identifies input errors from a puzzle; separate of handling a valid puzzle
class PuzzleReader
{
public:
   // Try to open the file, error if cannot
   bool read(const std::string &filename, SudokuGrid &grid, std::string &error) const
   {
       std::ifstream in(filename);
       if (!in)
       {
           error = "cannot open file '" + filename + "'";
           return false;
       }
       SudokuGrid parsed;
       std::string line;
       int rowsRead = 0;
       int lineNumber = 0;

       // Read one line at a time and advance the line number
       while (std::getline(in, line))
       {
           ++lineNumber;
           if (!line.empty() && line.back() == '\r')
           {
               line.pop_back();
           }
           std::istringstream tokens(line);
           // The initial reader stores every token from this row before validating
it.
           std::vector<std::string> fields;
           std::string token;
           // Read each token and add copy to rowss
           while (tokens >> token)
           {
```

```
                fields.push_back(token);
            }

            // reject data after nine rows (ignoring blank rows)
            // Return an error before changes if failed
            if (fields.empty())
            {
                continue;
            }

            if (rowsRead == SudokuGrid::SIZE)
            {
                error = "extra non-blank data at line " + std::to_string(lineNumber) +
" (more than 9 rows)";
                return false;
            }

            if (static_cast<int>(fields.size()) != SudokuGrid::SIZE)
            {
                error = "line " + std::to_string(lineNumber) + " has " +
std::to_string(fields.size()) + " tokens; expected 9";
                return false;
            }

            // Read each column token; error if not one digit or underscore
            // Store underscores as zero and digits as their value
            for (int c = 0; c < SudokuGrid::SIZE; ++c)
            {
                const std::string &f = fields[c];
                if (f.size() != 1)
                {
                    error = "line " + std::to_string(lineNumber) + ", column " +
std::to_string(c + 1) + ": invalid token '" + f + "'";
                    return false;
                }
                char ch = f[0];
                if (ch == '_')
                {
                    parsed.set(rowsRead, c, 0);
                }
                else if (ch >= '1' && ch <= '9')
                {
                    parsed.set(rowsRead, c, ch - '0');
                }
                else
                {
                    error = "line " + std::to_string(lineNumber) + ", column " +
std::to_string(c + 1) + ": invalid token '" + f + "'";
                    return false;
                }
            }
            ++rowsRead;
        }

        // Check for read errors or missing rows, then accept the board
        if (in.bad())
        {
            error = "I/O error while reading '" + filename + "'";
            return false;
        }

        if (rowsRead != SudokuGrid::SIZE)
        {
```

```
            error = "found " + std::to_string(rowsRead) + " puzzle rows; expected 9";
            return false;
        }

        grid = parsed;
        return true;
    }
};

// Backtrack through a copy and print solutions as found
class SudokuSolver
{
public:
    // Copy the puzzle and reset solution count
    // Track used digits in each row, column and box; repeated clues are conflicts
    SudokuSolver(const SudokuGrid &puzzle, std::ostream &out)
        : work_(puzzle),
          out_(out),
          solutionCount_(0),
          cluesConsistent_(true)
    {
        rowUsed_.fill(0);
        colUsed_.fill(0);
        boxUsed_.fill(0);
        for (int r = 0; r < SudokuGrid::SIZE; ++r)
        {
            for (int c = 0; c < SudokuGrid::SIZE; ++c)
            {
                int d = work_.get(r, c);
                if (d == 0)
                {
                    continue;
                }
                int bit = 1 << (d - 1);
                int b = boxIndex(r, c);
                if ((rowUsed_[r] & bit) || (colUsed_[c] & bit) || (boxUsed_[b] & bit))
                {
                    cluesConsistent_ = false;
                }
                rowUsed_[r] |= bit;
                colUsed_[c] |= bit;
                boxUsed_[b] |= bit;
            }
        }
    }

    // Search if clues do not conflict; return number of solutions
    long long solveAll()
    {
        if (cluesConsistent_)
        {
            search();
        }
        return solutionCount_;
    }

    // Return whether clues are consistent
    bool cluesConsistent() const
    {
        return cluesConsistent_;
    }

private:
```

```
// Find which of the nine boxes contains this cell
static int boxIndex(int r, int c)
{
    return (r / 3) * 3 + (c / 3);
}

// Combine used digits, invert and keep nine bits
// The remaining bits represent legal digits for this cell
int candidates(int r, int c) const
{
    int used = rowUsed_[r] | colUsed_[c] | boxUsed_[boxIndex(r, c)];
    return ~used & 0x1FF;
}

// Count choices by clearing one set bit at a time
static int bitCount(int mask)
{
    int count = 0;
    while (mask)
    {
        mask &= mask - 1;
        ++count;
    }
    return count;
}

// Find the empty cell with the fewest choices; start with none selected
void search()
{
    int bestRow = -1;
    int bestCol = -1;
    int bestMask = 0;
    int bestCount = 10;

    // Skip filled cells and track the empty cell with the fewest legal digits
    // Backtrack if any empty cell has no choices
    for (int r = 0; r < SudokuGrid::SIZE; ++r)
    {
        for (int c = 0; c < SudokuGrid::SIZE; ++c)
        {
            if (work_.get(r, c) != 0)
            {
                continue;
            }
            int mask = candidates(r, c);
            int count = bitCount(mask);
            if (count < bestCount)
            {
                bestRow = r;
                bestCol = c;
                bestMask = mask;
                bestCount = count;
                if (count == 0)
                {
                    return;
                }
            }
        }
    }

    // No blanks means solved; count and print this board
    // Return to try other solutions
    if (bestRow == -1)
```

```
        {
            ++solutionCount_;
            out_ << "Solution " << solutionCount_ << ":\n";
            work_.print(out_);
            out_ << '\n';
            return;
        }

        // Try legal digits 1-9 in the chosen cell
        // Mark the digit used in its row, column and box, then recurse
        int b = boxIndex(bestRow, bestCol);
        for (int d = 1; d <= SudokuGrid::SIZE; ++d)
        {
            int bit = 1 << (d - 1);
            if (!(bestMask & bit))
            {
                continue;
            }
            work_.set(bestRow, bestCol, d);
            rowUsed_[bestRow] |= bit;
            colUsed_[bestCol] |= bit;
            boxUsed_[b] |= bit;

            search();

            // Clear the digit from the masks and cell before the next attempt
            boxUsed_[b] &= ~bit;
            colUsed_[bestCol] &= ~bit;
            rowUsed_[bestRow] &= ~bit;
            work_.set(bestRow, bestCol, 0);
        }
    }

    // Store working board, output, used-digit masks, solution count and clue status
    SudokuGrid work_;
    std::ostream &out_;
    std::array<int, SudokuGrid::SIZE> rowUsed_;
    std::array<int, SudokuGrid::SIZE> colUsed_;
    std::array<int, SudokuGrid::SIZE> boxUsed_;
    long long solutionCount_;
    bool cluesConsistent_;
};

// Read, solve and print each file separately
class SudokuApplication
{
public:
    // Use given filenames, or puzzle1.txt thru puzzle5.txt by default
    SudokuApplication(int argc, char *argv[])
    {
        if (argc > 1)
        {
            for (int i = 1; i < argc; ++i)
            {
                files_.push_back(argv[i]);
            }
        }
        else
        {
            for (int i = 1; i <= 5; ++i)
            {
                files_.push_back("puzzle" + std::to_string(i) + ".txt");
            }
```

```
        }
    }

    // Process files in order; remember errors while continuing to later files
    // Return 1 for an input error, otherwise 0
    int run()
    {
        bool anyError = false;
        for (std::size_t i = 0; i < files_.size(); ++i)
        {
            if (i > 0)
            {
                std::cout << '\n';
            }
            if (!processFile(files_[i]))
            {
                anyError = true;
            }
        }
        return anyError ? 1 : 0;
    }

private:
    // Print filename and read into a fresh board
    // Report input errors separately from a valid puzzle with no solution
    bool processFile(const std::string &filename)
    {
        std::cout << "===== " << filename << " =====\n";

        SudokuGrid original;
        std::string error;
        if (!reader_.read(filename, original, error))
        {
            std::cout << "ERROR: " << error << '\n';
            std::cerr << "ERROR (" << filename << "): " << error << '\n';
            return false;
        }

        // Print original puzzle, then solve a copy
        // Report conflicting clues, all solutions and their count, or no solution
        std::cout << "Original puzzle:\n";
        original.print(std::cout);
        std::cout << '\n';

        SudokuSolver solver(original, std::cout);
        if (!solver.cluesConsistent())
        {
            std::cout << "Note: the given clues conflict (a digit repeats in a row, "
                         "column, or 3x3 box).\n";
        }
        long long count = solver.solveAll();

        if (count == 0)
        {
            std::cout << "No solution found\n";
        }
        else
        {
            std::cout << "Total solutions: " << count << '\n';
        }
        return true;
    }
```

```
    // Store filenames and file reader
    std::vector<std::string> files_;
    PuzzleReader reader_;
};

// Create the app and return its exit status
int main(int argc, char *argv[])
{
    SudokuApplication app(argc, argv);
    return app.run();
}
```

ii. Gemini's initial result (commented)

```
/*
EECS 348 Assignment 4
Sudoku solver: Ingest a 9x9 sudoku puzzle and return all solutions
Input: puzzle1.txt thru puzzle5.txt. Or dedicated filenames
Output: Filename, the original board, errors, and the possible solutions
Gemini 3.8 Flash (Low)
Matthew Bisbee
10/15/2026
*/

#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <cctype>

class SudokuGrid {
public:
    // Use a 9x9 board with 3x3 boxes
    static const int GRID_SIZE = 9;
    static const int BOX_SIZE = 3;

    // Start with zeros and no cells marked as clues
    SudokuGrid() {
        cells.assign(GRID_SIZE, std::vector<int>(GRID_SIZE, 0));

        isClue.assign(GRID_SIZE, std::vector<bool>(GRID_SIZE, false));
    }

    // Return stored value at that location
    int getCell(int r, int c) const {
        return cells[r][c];
    }

    // Store value and whether it is an original clue
    void setCell(int r, int c, int val, bool asClue) {
        cells[r][c] = val;
        isClue[r][c] = asClue;
    }

    // Change the value without changing its clue flag
    void setCellValueOnly(int r, int c, int val) {
        cells[r][c] = val;
    }

    // Return whether this cell was an original clue
    bool isOriginalClue(int r, int c) const {
        return isClue[r][c];
    }

    // Check each row for repeated clues, ignoring blanks
```

```
// Reset the seen list for each row and mark digits as found
bool isConsistentClues() const {
    for (int r = 0; r < GRID_SIZE; ++r) {
        std::vector<bool> seen(10, false);
        for (int c = 0; c < GRID_SIZE; ++c) {
            int val = cells[r][c];
            if (val != 0) {
                if (seen[val]) {
                    return false;
                }
                seen[val] = true;
            }
        }
    }

    // Check each column for repeated clues
    for (int c = 0; c < GRID_SIZE; ++c) {
        std::vector<bool> seen(10, false);
        for (int r = 0; r < GRID_SIZE; ++r) {
            int val = cells[r][c];
            if (val != 0) {
                if (seen[val]) {
                    return false;
                }
                seen[val] = true;
            }
        }
    }

    // Check each 3x3 box for repeated clues
    // Return true if rows, columns and boxes have no conflicts
    for (int boxRow = 0; boxRow < GRID_SIZE; boxRow += BOX_SIZE) {
        for (int boxCol = 0; boxCol < GRID_SIZE; boxCol += BOX_SIZE) {
            std::vector<bool> seen(10, false);
            for (int r = 0; r < BOX_SIZE; ++r) {
                for (int c = 0; c < BOX_SIZE; ++c) {
                    int val = cells[boxRow + r][boxCol + c];
                    if (val != 0) {
                        if (seen[val]) {
                            return false;
                        }
                        seen[val] = true;
                    }
                }
            }
        }
    }

    return true;
}

// Check if this digit is already in the row, column or box
// Return false on a match; otherwise the placement is legal
bool isLegalCandidate(int row, int col, int val) const {
    for (int c = 0; c < GRID_SIZE; ++c) {
        if (cells[row][c] == val) {
            return false;
        }
    }

    for (int r = 0; r < GRID_SIZE; ++r) {
        if (cells[r][col] == val) {
            return false;
```

```
            }
        }

        // Find the top left of this box, then check its nine cells
        int startRow = (row / BOX_SIZE) * BOX_SIZE;
        int startCol = (col / BOX_SIZE) * BOX_SIZE;
        for (int r = 0; r < BOX_SIZE; ++r) {
            for (int c = 0; c < BOX_SIZE; ++c) {
                if (cells[startRow + r][startCol + c] == val) {
                    return false;
                }
            }
        }

        return true;
    }

    // Print each row in order, with spaces between cells
    // Show blanks as underscores if requested, otherwise as zero
    void printGrid(std::ostream& out, bool showBlanks) const {
        for (int r = 0; r < GRID_SIZE; ++r) {
            for (int c = 0; c < GRID_SIZE; ++c) {
                if (cells[r][c] == 0) {
                    if (showBlanks) {
                        out << '_';
                    } else {
                        out << '0';
                    }
                } else {
                    out << cells[r][c];
                }
                if (c + 1 < GRID_SIZE) {
                    out << ' ';
                }
            }
            out << '\n';
        }
    }

private:
    // Store cell values and clue flags
    std::vector<std::vector<int>> cells;
    std::vector<std::vector<bool>> isClue;
};

// Backtrack through placements and save all solutions
class SudokuSolver {
public:
    // Start with an empty solution list
    SudokuSolver() {}

    // Clear earlier solutions and check clues before searching from the first cell
    void solveAll(SudokuGrid& grid) {
        solutions.clear();

        if (!grid.isConsistentClues()) {
            return;
        }

        search(grid, 0, 0);
    }

    // Return saved solutions without copying the list
```

```
    const std::vector<SudokuGrid>& getSolutions() const {
        return solutions;
    }

    // Return solution count
    size_t getSolutionCount() const {
        return solutions.size();
    }

private:
    // Store all solutions until printed
    std::vector<SudokuGrid> solutions;

    // Save the board when all rows are filled, then return for more attempts
    // Otherwise find the next cell and skip original clues
    void search(SudokuGrid& grid, int row, int col) {
        if (row == SudokuGrid::GRID_SIZE) {
            solutions.push_back(grid);
            return;
        }

        int nextRow = (col + 1 == SudokuGrid::GRID_SIZE) ? (row + 1) : row;
        int nextCol = (col + 1 == SudokuGrid::GRID_SIZE) ? 0 : (col + 1);

        if (grid.getCell(row, col) != 0) {
            search(grid, nextRow, nextCol);
            return;
        }

        // Try digits 1-9; place legal choices and recurse
        // Clear the cell after returning so the next attempt starts unchanged
        for (int candidate = 1; candidate <= 9; ++candidate) {
            if (grid.isLegalCandidate(row, col, candidate)) {
                grid.setCellValueOnly(row, col, candidate);

                search(grid, nextRow, nextCol);

                grid.setCellValueOnly(row, col, 0);
            }
        }
    }
};

// Read each file and print its puzzle and solutions
class SudokuApplication {
public:
    // Use given filenames, or the five defaults
    // Process in order, separating results with blank lines
    // This version returns 0 even if a file reports an error
    int run(int argc, char* argv[]) {
        std::vector<std::string> fileList;

        if (argc > 1) {
            for (int i = 1; i < argc; ++i) {
                fileList.push_back(argv[i]);
            }
        } else {
            fileList.push_back("puzzle1.txt");
            fileList.push_back("puzzle2.txt");
            fileList.push_back("puzzle3.txt");
            fileList.push_back("puzzle4.txt");
            fileList.push_back("puzzle5.txt");
        }
```

```
        for (size_t i = 0; i < fileList.size(); ++i) {
            processPuzzleFile(fileList[i]);
            if (i + 1 < fileList.size()) {
                std::cout << "\n";
            }
        }

        return 0;
    }

private:
    // Print filename and try to open it; report errors and skip if failed
    void processPuzzleFile(const std::string& filename) {
        std::cout << "Puzzle: " << filename << "\n";

        std::ifstream inFile(filename.c_str());
        if (!inFile.is_open()) {
            std::cout << "Error: Could not open puzzle file " << filename << "\n";
            return;
        }

        // Read file tokens; require 81 entries before filling the board
        SudokuGrid originalGrid;
        std::vector<std::string> tokens;
        std::string token;

        while (inFile >> token) {
            tokens.push_back(token);
        }
        inFile.close();

        if (tokens.size() != 81) {
            std::cout << "Error: Malformed puzzle input in " << filename
                      << " (expected 81 tokens, found " << tokens.size() << ")\n";
            return;
        }

        // Find row and column from the token index
        // Store underscores as blanks and digits as clues; error on other tokens
        for (int idx = 0; idx < 81; ++idx) {
            int r = idx / 9;
            int c = idx % 9;
            const std::string& tok = tokens[idx];

            if (tok == "_") {
                originalGrid.setCell(r, c, 0, false);
            } else if (tok.length() == 1 && tok[0] >= '1' && tok[0] <= '9') {
                int digit = tok[0] - '0';
                originalGrid.setCell(r, c, digit, true);
            } else {
                std::cout << "Error: Invalid token '" << tok << "' at cell ("
                          << r << ", " << c << ") in " << filename << "\n";
                return;
            }
        }

        // Print original puzzle, then solve a separate copy with a fresh solver
        std::cout << "Original puzzle:\n";
        originalGrid.printGrid(std::cout, true);

        SudokuGrid workingGrid = originalGrid;
```

```
        SudokuSolver solver;
        solver.solveAll(workingGrid);

        // Print no solution if empty; otherwise print count and all saved boards
        const std::vector<SudokuGrid>& solutions = solver.getSolutions();
        if (solutions.empty()) {
            std::cout << "No solution found\n";
        } else {
            std::cout << "Found " << solutions.size() << " solution(s):\n";
            for (size_t s = 0; s < solutions.size(); ++s) {
                std::cout << "--- Solution " << (s + 1) << " ---\n";
                solutions[s].printGrid(std::cout, false);
            }
        }
    }
};

// Create the app
int main(int argc, char* argv[]) {
    SudokuApplication app;
    return app.run(argc, argv);
}
```

e. Code analysis

i. Claude

1. Correctness

The original program did all 5 puzzles properly on the cycle server. It found both solutions for puzzle 5 and found no solution for puzzle 4. If an invalid file was fed, it was handled separately. The program did not check for failed output, so a success could be logged without results.

2. Execution Time

The search tries all possible combinations of possible digits and backtracks when options fail. With some number of empty cells X, a worst case scenario could be O(9^X). Claude chose to target the cell with the fewest legal options left, which could minimize the number of invalid boards tried. No practical speed advantage was felt when running as compared to Gemini.

3. Space Complexity

The board array was fixed in size. The recursion however grows with the number of empty cells. Solutions are never stored, just printed, so more solutions doesn't mean more memory used. See below how this differs from Gemini.

4. Maintainability

The grid, file reader, and solver are all logically separated. Them having distinct roles is helpful and makes clear where inputs are handled versus where solving happens.

ii. Gemini

1. Correctness

Gemini also solved the 5 given puzzles without issue, including the 2 solutions puzzle and the unsolvable one. It passed the same valid inputs, and improper inputs produced an error message. However, the overall program still gave a success status, making it unclear if the entire run was a success or not.

2. Execution Time

It uses the same approach as mentioned above for Claude. Cells are checked in order of rows and scans each cell for valid numbers. It also copies completed boards before printing them. Again, couldn't establish an obvious time advantage for either program.

3. Space Complexity

Each completed board is stored until the search is finished. For example: with 3 solutions, that storage would be O(3), with each of the 3 having a fixed board size. So unlike Claude, memory usage is increasing with the number of solutions possible.

4. Maintainability

The very direct row by row search makes the program easy to trace. Not everything inside the solver seemed critical; there was some fluff. There is a separation of roles, similar to Claude's. However, Claudes' separation may be slightly more clear.

f. Selection

I selected Claude's implementation. Both performed well on the 5 sample puzzles, so the choice was between their internal differences. Claude printed solutions without storing all of them in memory and better separated reading files from solving them. Gemini combined file *reading* with file *displaying* into the same class, while Claude came up with a separate reader. I think Claude's version was ultimately a better starting point that I would want to make changes to.

g. Improvements

i. Correctness

The program was revised to check if an output was successfully written and error if it is not. This was also tested and verified on Cycle. Exception handling was also added. This revision still passes the same tests.

ii. Execution Time

The file reader can check tokens directly now, rather than making and copying a new list for every row. The solver's steps remain largely unchanged, so a minimal speed improvement should be expected; ultimately, some unnecessary input handling was shaved off.

iii. Space Complexity

The temporary list of row tokens was removed entirely, so some memory usage during runs was optimized. The board representation and recursion are unchanged.

iv. Maintainability

Token validation now has a separate helper function. Input/output errors, and exceptions are now all handled in separate places instead of clumped together. Each exit code is now specified so the results are clearer. The code is longer overall, but the added checks are helpful and the main solving logic is the same as before.